

AI Agent Engineer

Async Document Processing Agent Service · 6-Hour Take-Home Assessment

TIME LIMIT	SUBMISSION	LLM API	STACK
6 hours	GitHub + Video	Use your own	Your choice

Background

Robotic Imaging needs an internal service that processes incoming documents at scale. Teams across the company submit documents (contracts, reports, specs) and need them analyzed by an AI agent. The service must be **production-shaped**: reliable, observable, and deployable.

This assessment is intentionally under-specified in places. We want to see how you make tradeoffs, what you prioritize, and how you reason about ambiguity. **Completeness is not the goal — judgment is.**

How we evaluate

We weight **system design (30%)**, **code quality (20%)**, **DevOps (20%)**, **agent quality (15%)**, and **thinking & documentation (15%)**. An honest, well-documented partial submission beats a sprawling, fragile complete one.

What to Build

1. REST API Endpoints

Method	Endpoint	Purpose
POST	/jobs	Submit a document URL + analysis type (<i>summary / extraction / classification</i>)
GET	/jobs/:id	Check status and retrieve results
GET	/jobs	List jobs with filters (status, date range, type)
GET	/healthz	Liveness check that actually verifies dependencies (DB, queue)
GET	/metrics	Ops metrics: jobs processed, error rate, avg latency, LLM token spend

2. Async Processing

Synchronous processing is an automatic fail. POST /jobs must return immediately with a job ID. A worker picks up the job from a queue and processes it in the background.

3. AI Agent (per job)

- Fetch the document from the URL
- Run the requested analysis using an LLM
- **Extraction** → structured JSON matching a schema you define
- **Summary** → multi-section summary with confidence scores
- **Classification** → category + reasoning + confidence
- **Self-validate** output before marking the job complete

4. Real Failure Modes to Handle

- LLM API timeouts and rate limits → retry with exponential backoff
- Malformed documents → graceful failure with a useful error
- Worker crashes mid-job → the job must **not** be lost
- Duplicate submissions → idempotent POST /jobs
- Cost overruns → enforce a per-job token budget

5. Persistence

Use a database (SQLite is fine, Postgres is better) to persist:

- Jobs, statuses, inputs, outputs, error logs
- Token usage per job (for cost tracking)
- Audit trail of every state transition

6. Observability

- **Structured logs** (JSON) with correlation IDs flowing API → queue → worker
- **Metrics endpoint** exposing counters and histograms
- **One alerting condition** documented (when would you page someone, and why?)

7. Deployable

- docker - compose up brings the entire system online (API + worker + queue + DB)
- README explaining how to run, test, and what each component does
- At least **one integration test** that submits a job end-to-end and verifies output
- All configuration via environment variables — no hardcoded secrets or URLs

8. Load Test

Demonstrate the system handles **50 concurrent job submissions** without dropping any. Include the results (numbers, screenshots, or logs) in your README.

Resources You'll Need

You are responsible for setting up your own working environment:

- **LLM API access** — use your own key (Anthropic, OpenAI, or any provider you prefer). Free trial credits are sufficient for this assignment.
- **Test documents** — pick 3–5 public PDFs of your choice (contracts, research papers, reports, invoices — variety helps). Document your choices in the README.

- **Tooling** — language, framework, queue, database are all your call. Justify your picks in the README.

Submission Requirements

Item	Description
GitHub repo	Full source. Public or invite us as collaborators. Include git history — we look at it.
README.md	Setup instructions, design decisions, what you'd build next, what you cut and why .
ARCHITECTURE.md	System diagram (Excalidraw, Mermaid, hand-drawn photo — anything legible) + component descriptions.
Video walkthrough	5–8 minute screen recording: show it running, walk the architecture, demo a failure scenario, explain one tradeoff you debated.

Evaluation Rubric

Dimension	Weight	What we look for
System design	30%	Real async architecture, idempotency, explicit job state machine, sane queue choice, failure isolation
Code quality	20%	Modular, typed, tested, readable. Distinct error types for distinct failures. 12-factor config.
DevOps	20%	Fresh-clone docker-compose up works. Real healthcheck. Traceable logs. Env-based config.
Agent quality	15%	Structured outputs (JSON mode / function calling). Self-validation that <i>actually</i> validates.
Thinking & docs	15%	Explicit tradeoffs. Honest "what I cut". Specific "what I'd do next" — no platitudes.

Signals We Care About

■ Green flags	■ Red flags
---------------	-------------

- Mentions prompt injection unprompted
- Correlation ID flows end-to-end in logs
- Job state machine documented (diagram or enum)
- Schema has version or metadata JSON for future flexibility
- Small CLI for debugging (e.g. `inspect-job <id>`)
- Honest, specific "what I'd do next" list
- Load test reveals a bottleneck — and they document it

- "Async" is just `asyncio.create_task` in the API process
- Healthcheck returns 200 without checking anything
- No idempotency on `POST /jobs`
- Self-validation = asking the LLM "is this correct?" and trusting
- README has no "what I cut" section
- Hardcoded secrets / URLs
- Worker crash loses the job

A note on philosophy

We are hiring for production agent engineering, not prompt-tutorial completion. We would rather see a tightly-scoped 60% submission with sharp design judgment than a sprawling 100% submission held together with duct tape. **Make your tradeoffs explicit. Be honest about what you cut. Show us how you think.**